AWS Data Platform DevOps - krótki przegląd kodu Python

To zadanie ustne. Nie musisz otwierać edytora, pisać całej funkcji, implementować testów ani korzystać z SDK AWS. Na omówienie kodu przeznacz około czterech minut; pozostała część bloku Python dotyczy Twoich rzeczywistych doświadczeń z automatyzacją.

Polska wersja służy przygotowaniu do rozmowy. Zgodnie z ustalonym formatem sama rozmowa i wyjaśnianie toku rozumowania odbywają się po angielsku.

Poniższe rekordy reprezentują zadania przetwarzające dane:

```
jobs = [
    {"name": "load_customers", "status": "SUCCEEDED"},
    {"name": "load_orders", "status": "FAILED"},
    {"name": "load_products", "status": "FAILED"},
]
```

Każde wywołanie powinno zwracać wyłącznie nazwy nieudanych zadań z tego wywołania, niezależnie od wcześniejszych wywołań. Funkcja zawiera dwa błędy:

```
def failed_jobs(jobs, failed=[]):
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
        return failed
```

Pytania

1. Co funkcja zwróci dla przykładowych danych i dlaczego?
2. Jaka drobna zmiana sprawi, że sprawdzi wszystkie zadania przed zwróceniem wyniku?
3. Po wprowadzeniu tylko tej zmiany, co zwrócą dwa kolejne wywołania `failed_jobs(jobs)` dla przykładowych danych? Przyjmij świeżą definicję funkcji i brak innych wywołań. Wyjaśnij drugi błąd i sposób jego naprawy.
4. Po naprawieniu obu błędów, co powinna zwrócić dla pustych danych oraz gdy wszystkie zadania zakończyły się sukcesem?

Możesz wyjaśnić obie poprawki ustnie; wpisywanie kodu jest opcjonalne.

Przyjmij, że każdy rekord zawiera poprawne pola `name` i `status`. Nie musisz projektować walidacji, ponowień, logowania ani systemu monitorowania dla tego fragmentu.

Krótka rozmowa o doświadczeniu

Po omówieniu kodu opisz jedną automatyzację w Pythonie, nad którą osobiście pracowałeś: co robiła, jak obsługiwała błędy i jak sprawdzałeś ją przed wdrożeniem. W tej części również nie trzeba nic implementować.